3D Gaussian Splatting

How computers learn to see the world in 3D

Jaime Hernández Yauheni Yatsevich

March 2026

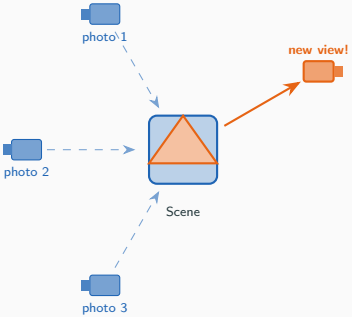
Universidad de Alicante

The Goal: See the World from Any Angle

Can a computer reconstruct a 3D scene just from photos?

1. Take photos from multiple angles
2. Build a 3D model
3. Render from any new viewpoint — in real time

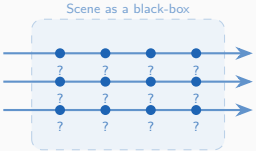
Applications
VR/AR · Film VFX · Robotics · Digital heritage



Input: photos from known angles ⟶ Output: any new angle

Two Approaches to the Same Problem

NeRF — *"Neural" approach*



Shoot rays → ask network at every point

Bottleneck

Thousands of neural-network queries **per pixel**

Training: 1–2 days Speed: <1 FPS

3D Gaussian Splatting — *"Explicit" approach*

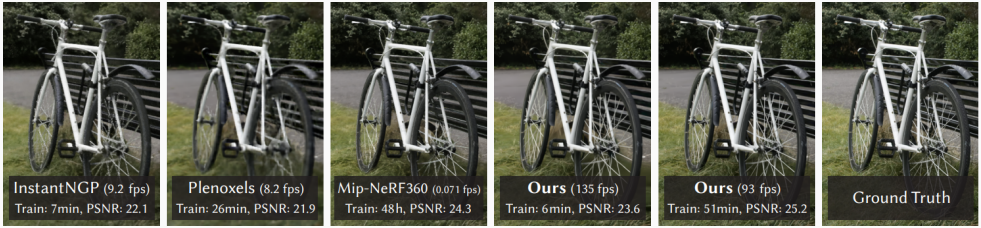
Millions of colored 3D blobs

Result

Just **project blobs** onto screen (GPU-fast)

Training: 20–40 min Speed: >**90 FPS**

3DGS: Photo-Realistic Results in Real Time



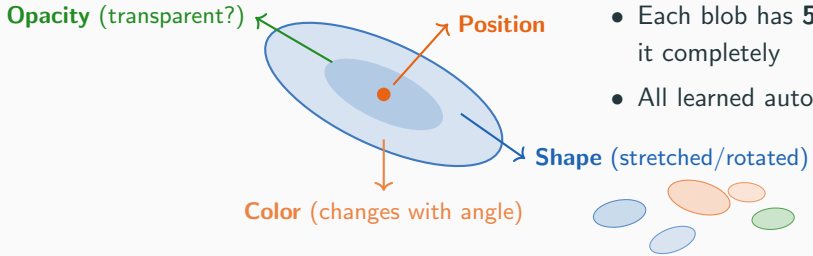
Real-time rendering
>**90 FPS** at 1080p

Fast training
20–40 min (vs. 1–2 days for NeRF)

Quality
Comparable to state-of-the-art NeRF

Building Block: The 3D Gaussian "Blob"

Each Gaussian is like a **colored, transparent soap bubble**:



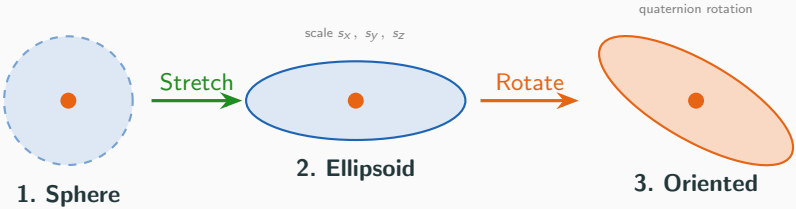
Each blob = one Gaussian primitive

A scene = millions of blobs

- Between **100,000 and 10 million** blobs per scene
- Each blob has **59 numbers** that define it completely
- All learned automatically from photos!

Blob Shape: Stretch and Rotate

Every blob can be any shape and orientation — like kneading clay



Why this is clever

Rotation & scale are stored separately, so the optimizer can freely adjust them without breaking math constraints.

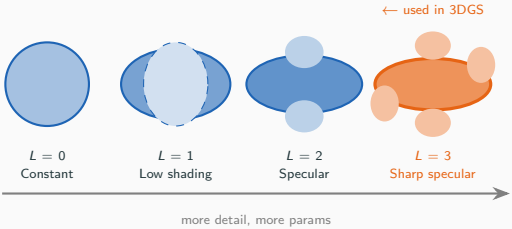
Different orientations in 3D space

Color that Changes with the Viewing Angle

Real surfaces look different depending on where you look from:

- A metallic ball has highlights
- Glass reflects the environment
- Wet surfaces are shiny from some angles

Spherical Harmonics capture this effect — like a flexible colour function on a sphere. Each blob stores **48 colour values** that together encode how colour changes with direction.

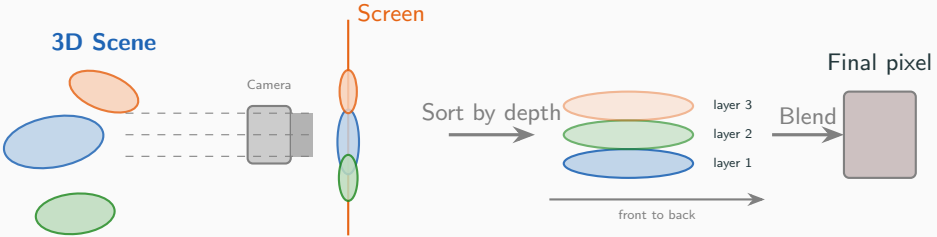


Training trick

Start simple ($L = 0$), unlock complexity every 1000 steps.

Rendering: Squashing Blobs onto the Screen

To make an image, each 3D blob is "splatted" (projected) onto the camera plane



Key insight

Unlike NeRF (which shoots rays), 3DGS **rasterizes** blobs directly onto the screen — exactly like how game engines render triangles. This is why it's so fast.

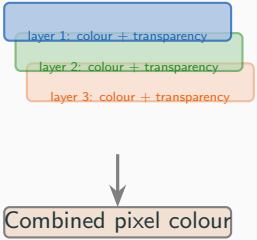
Layering Colours: Like Transparent Paint

Think of it like stacking transparent acetates:

1. Sort Gaussians **front to back**
2. Each layer contributes some colour and blocks some light behind it
3. Stop when the pixel is fully covered

Final pixel colour

$$\mathbf{C} = \sum_i \underbrace{\mathbf{c}_i}_{\text{colour}} \cdot \underbrace{\alpha_i}_{\text{opacity}} \cdot \underbrace{T_i}_{\text{how much light reaches layer } i}$$

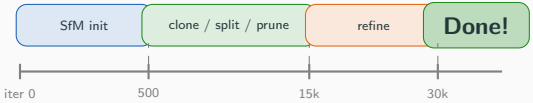


Same math as NeRF

This blending is *mathematically equivalent* to NeRF's ray integrals — just computed much faster.

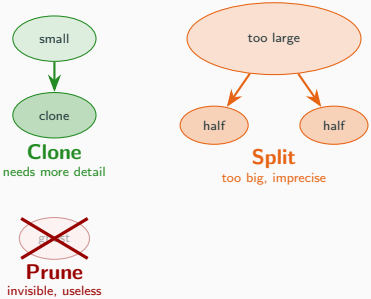
Training: The Scene Builds Itself

Start with a sparse point cloud, then refine:



Loss: rendered vs. real photo
Compare each rendered frame to the training photo. Adjust all blobs to reduce the error.

Three adaptive operations every 100 iterations:



Where It Struggles & Where It's Going

Current limitations:

- $\approx$**1 GB** High memory per scene
- $\times$ Bad if camera coverage is sparse
- $\sim$ Harder to edit semantically

Exciting future directions:

- $\rightarrow$ 4DGS — Dynamic scenes & video
- $\rightarrow$ Compressed Gaussians — smaller files
- $\rightarrow$ LiDAR init — better accuracy
- $\rightarrow$ Semantic editing — "change the chair"

Questions?

Jaime Hernández · Yauheni Yatsevich
Universidad de Alicante, March 2026